MINISTRY OF EDUCATION AND SCIENCE OF UKRAINE
NATIONAL TECHNICAL UNIVERSITY
“KHARKIV POLYTECHNIC INSTITUTE”
DEPARTMENT OF SOFTWARE ENGINEERING AND MANAGEMENT INFORMATION TECHNOLOGIES

# Report of laboratory work № 3

Discipline «Software Engineering»
Theme: Planning of System Activities

Executed by:
Student of group KN-226e
Budkov O. Yu.

Checked by:
Associate professor
Melnyk K. V.

Kharkiv
2026

**Theme:** planning of system activities.

**Goal:** to obtain basic experience and knowledge to work with a project tracking system.

**Tasks:**

– register on the free github.com account;

– create a new repository;

– add a new user as a collaborator to the repository;

– create milestones and issues according to the previous labs and assign tasks to the users of the team;

– work with the git repository;

– prepare the report of the work.

## Progress of work

### Task 3.1 Registering on github.com

GitHub is a web platform for version control and teamwork. It stores the repository of the project with the full history of changes and gives tools for planning the work: milestones, issues, labels and assignees. A free account was registered on https://github.com/signup (Figure 3.1).

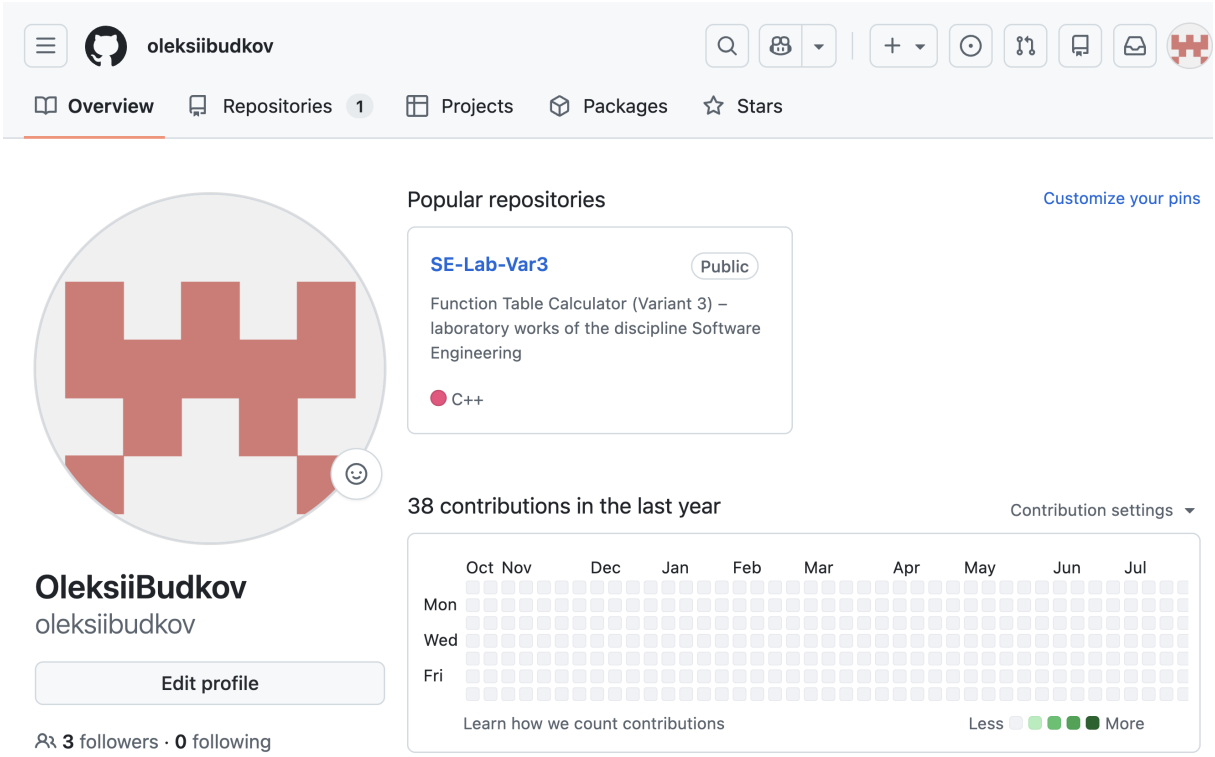



Figure 3.1 – Account on github.com

### Task 3.2 Creating a New Repository

A new public repository SE-Lab-Var3 was created with the button “New” (Figure 3.2). Its description is “Function Table Calculator (Variant 3) – laboratory works of the discipline Software Engineering”. The repository stores the C++ program, the reports and the diagrams of all labs (Figure 3.3); the file README.md describes the program and the structure of the repository.

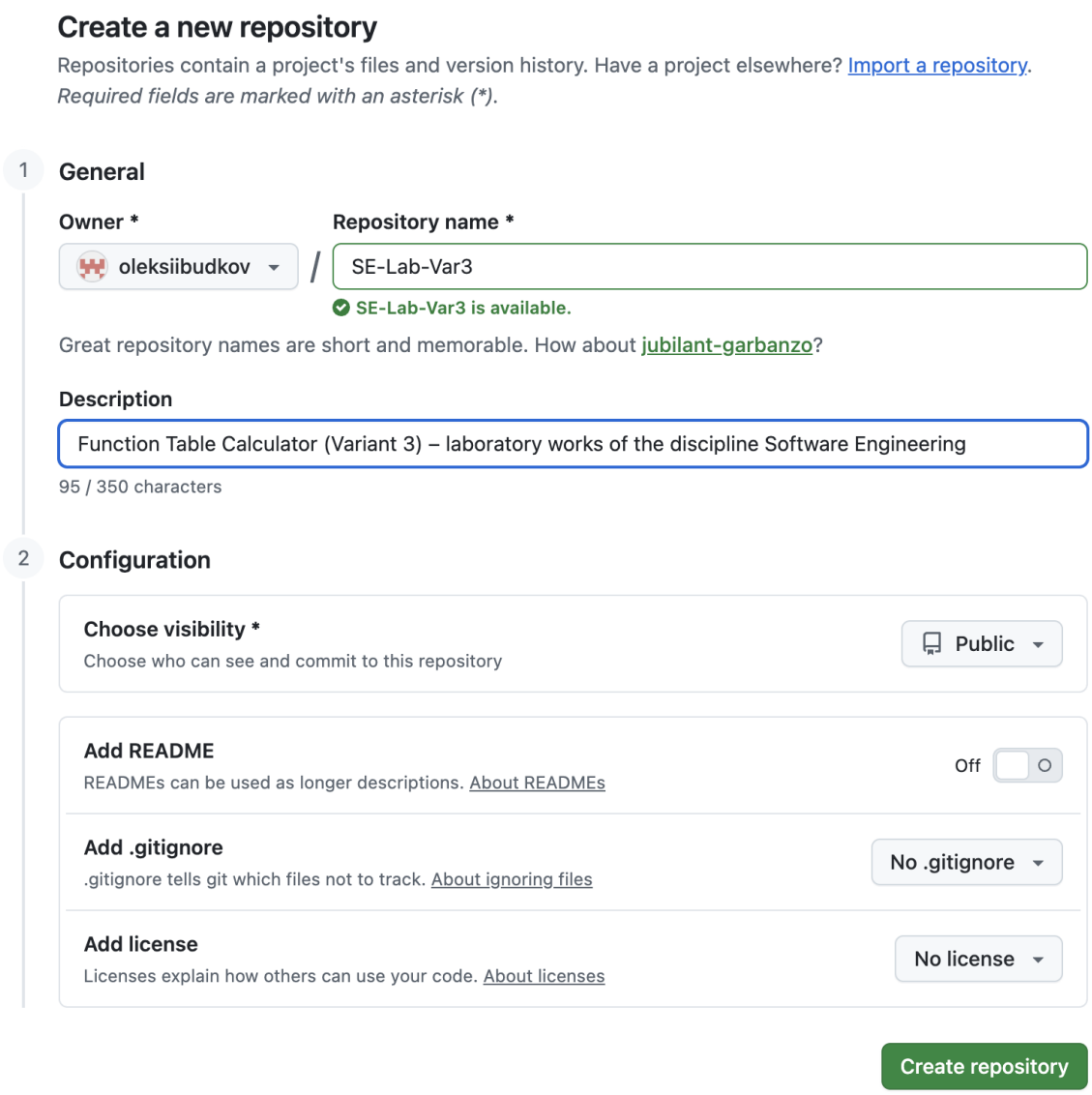



Figure 3.2 – Creating of a new repository

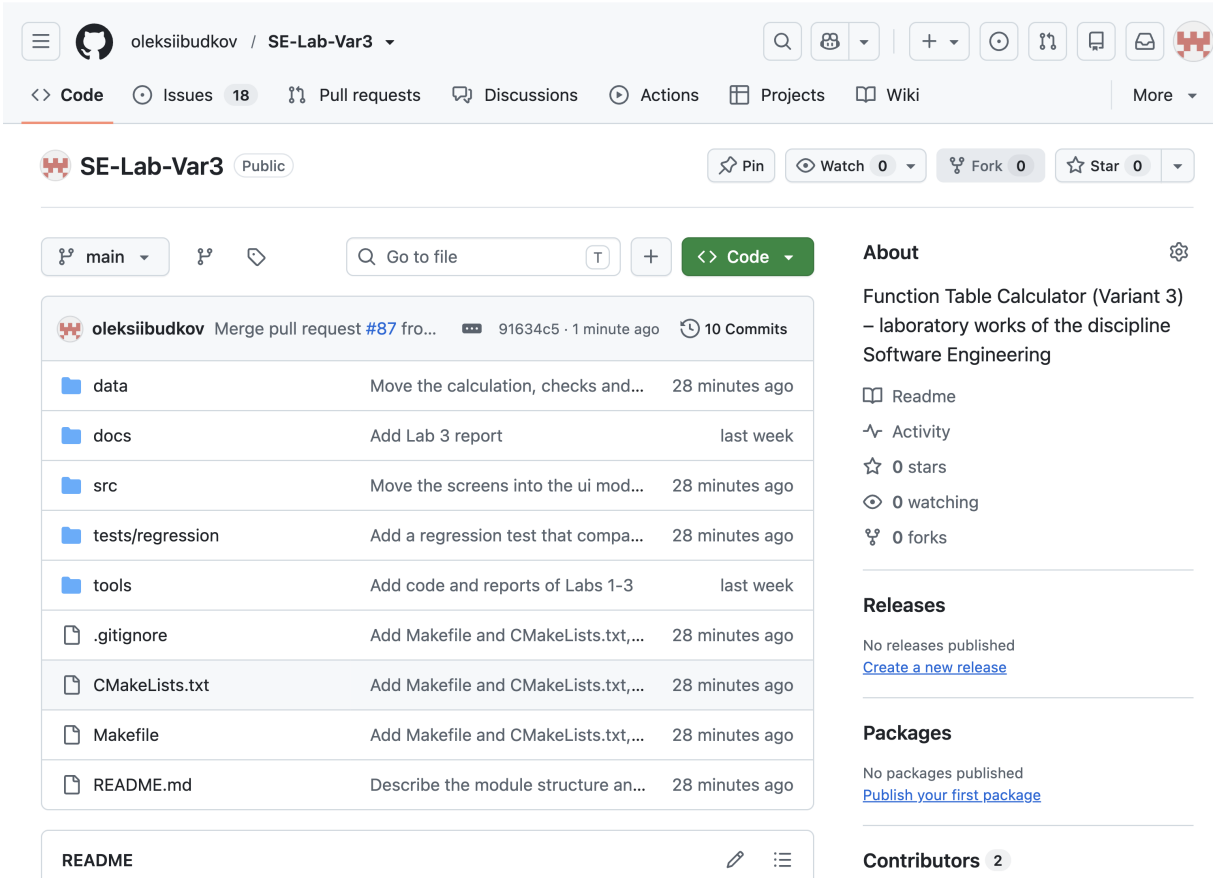



Figure 3.3 – The new repository on github.com

## Task 3.3 Adding a Collaborator

The project is developed by one person, so no second member was invited. A collaborator is invited on the page Settings → Collaborators → Add people (Figure 3.4): after the invitation is accepted, the collaborator can push changes to the repository and can be assigned to issues. All issues of the project are assigned to the author of the repository.

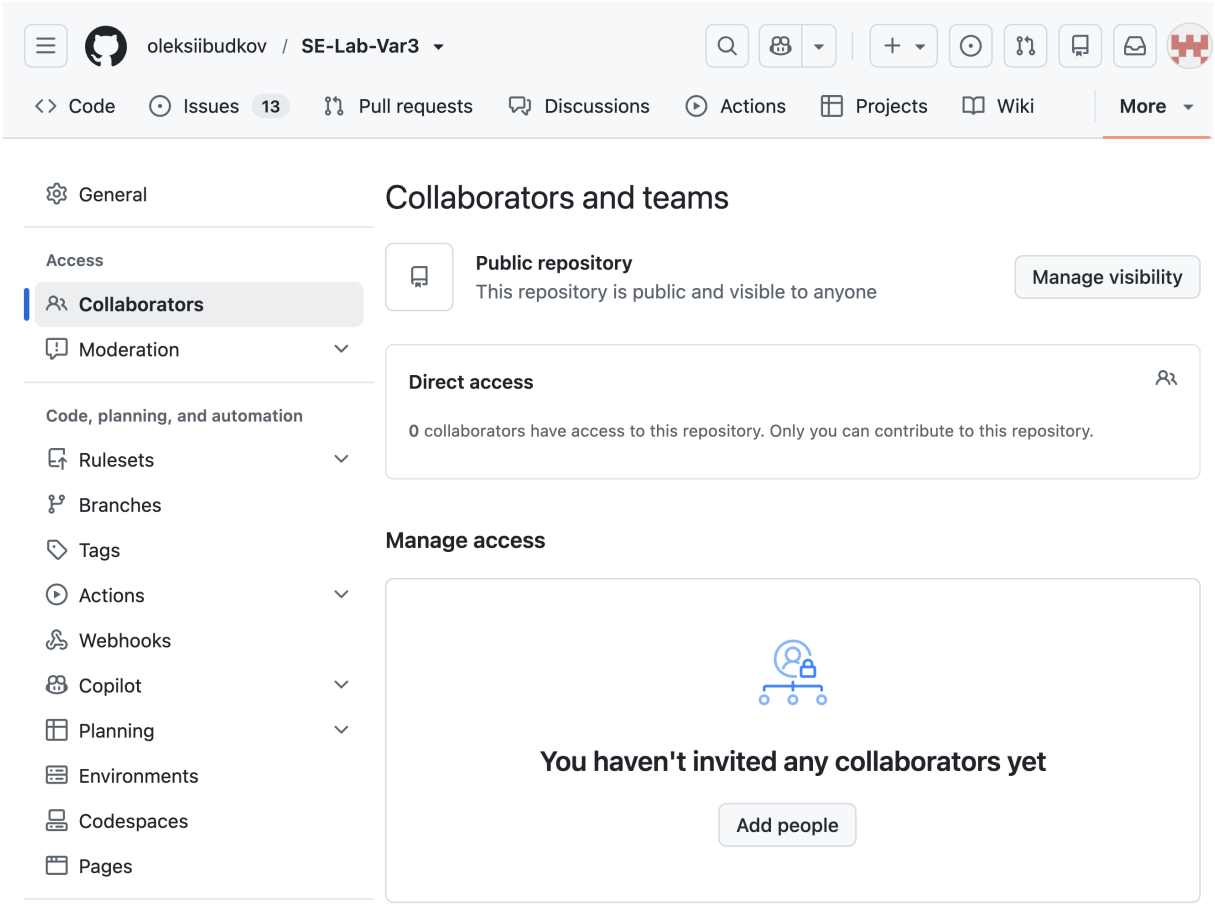



Figure 3.4 – Collaborators of the repository

## Task 3.4 Planning with Scrum: Milestones and Issues

The work is planned with Scrum. Instead of a plan of phases (analysis, coding, specification, testing), the project is a sequence of sprints of the same length — three weeks. Every sprint has a sprint goal and ends with an increment of the product that can be shown to the customer. All requirements are kept in the Product Backlog ordered by priority, and the work chosen for the current sprint is the Sprint Backlog. The project is made by one student, so one person has all three Scrum roles: Product Owner, Scrum Master and Developer.

Every Product Backlog item is a user story written as “As a <role>, I want <goal>, so that <benefit>”. A story has acceptance criteria, an estimate in story points (the Fibonacci numbers 1, 2, 3, 5, 8) and a MoSCoW priority: Must, Should or Could have. Labels show the type of an item — user story or task — and its priority (Figure 3.5). A story is created as an issue with these labels, the sprint (milestone) and the assignee (Figure 3.6). The technical work of a story is split into tasks; every task is a sub-issue of its story, so the story shows the progress of its tasks (Figure 3.7).

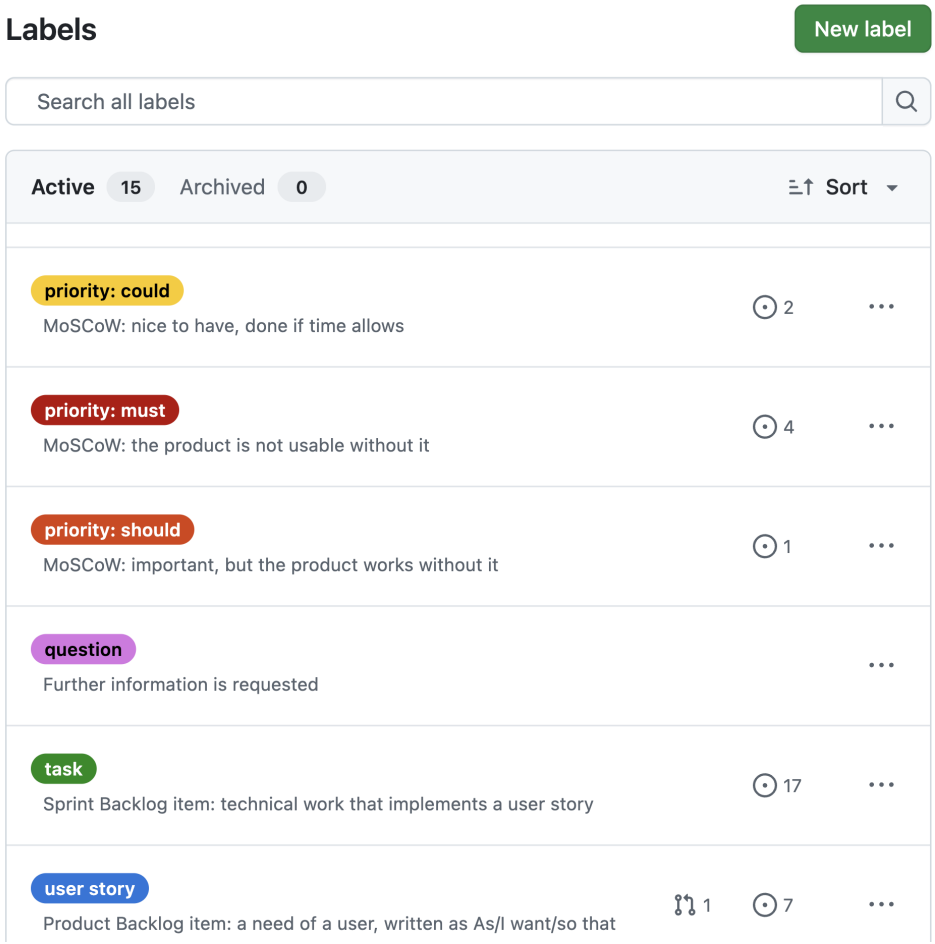



Figure 3.5 – Labels of the Product Backlog items

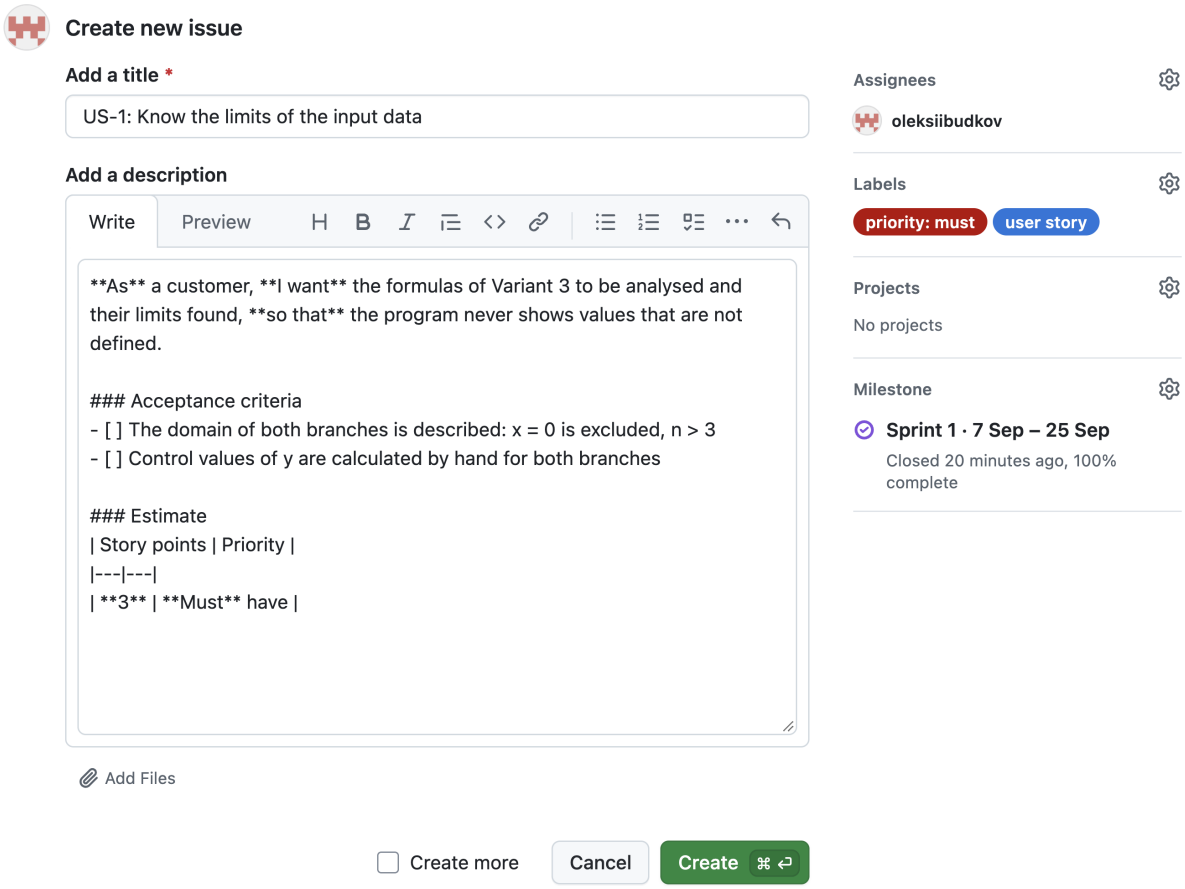



Figure 3.6 – Creating of a user story

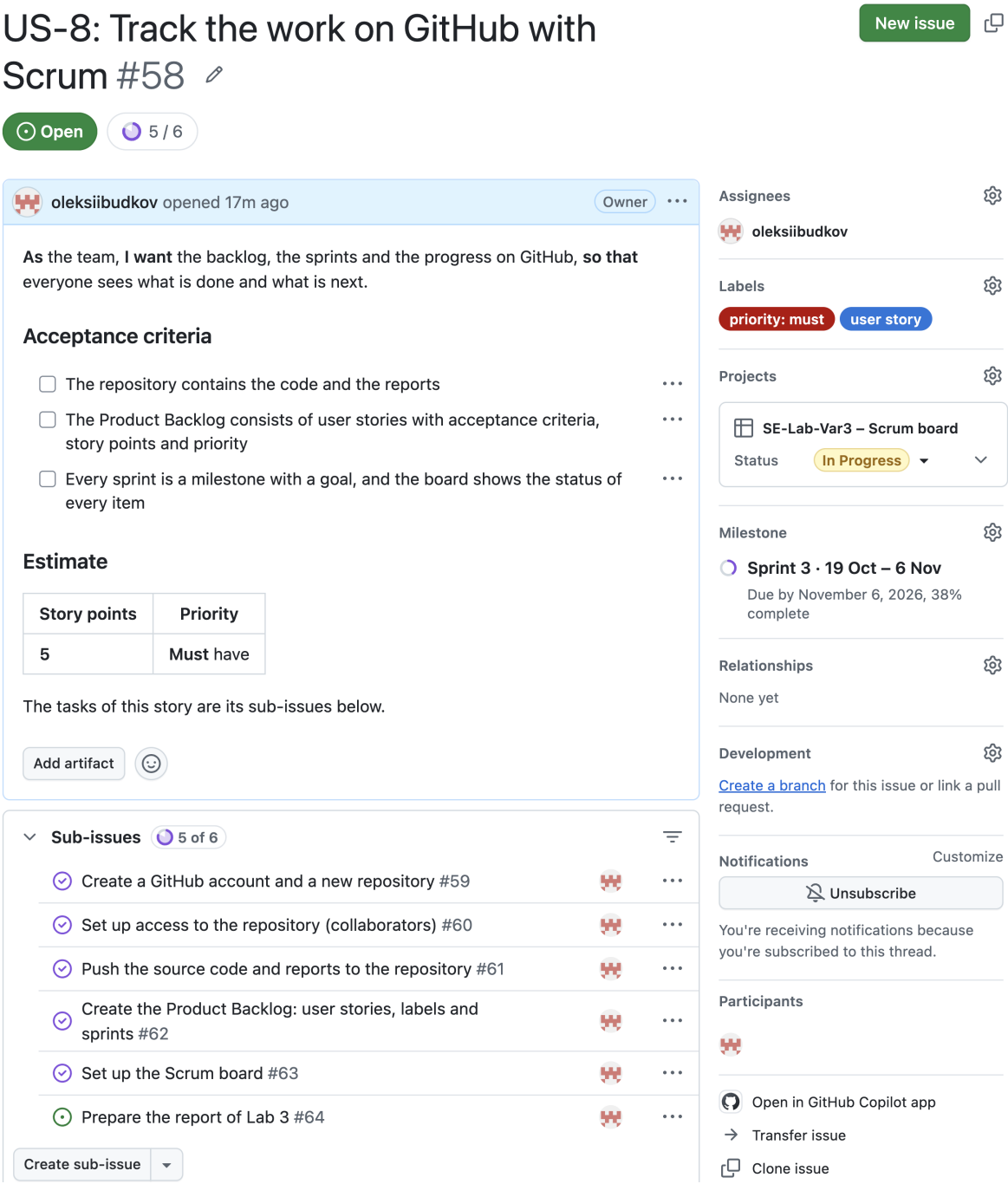



Figure 3.7 – User story with its tasks (sub-issues)

Every sprint is a milestone: the title has the dates of the sprint, the description has the sprint goal, and the due date is the last day of the sprint (Figure 3.8). Five sprints cover the semester (Figure 3.9); the deadlines of the laboratory works fall inside them. When a sprint ends, its milestone is closed and its description keeps the velocity — the story points done — and the main idea of the retrospective. Two stories that are not planned for a sprint yet stay in the Product Backlog.

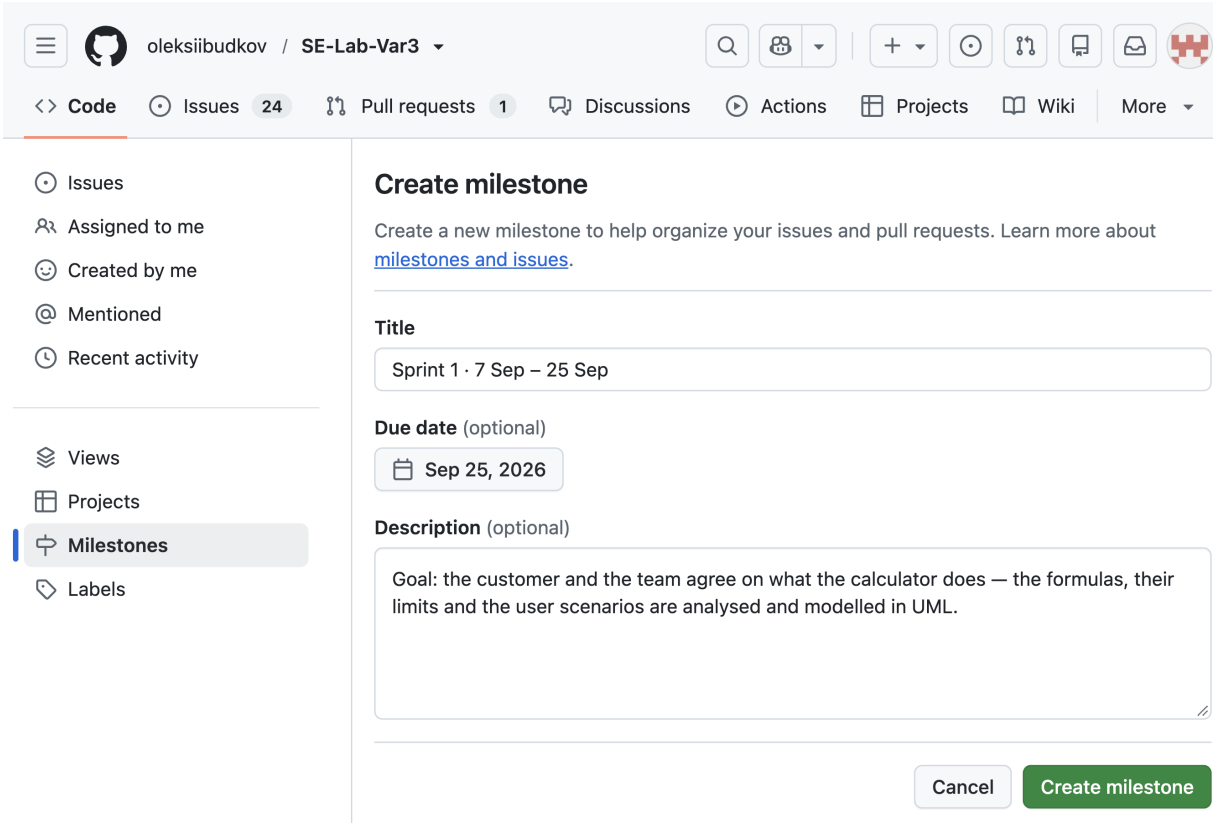



Figure 3.8 – Creating of a sprint (milestone with a sprint goal)

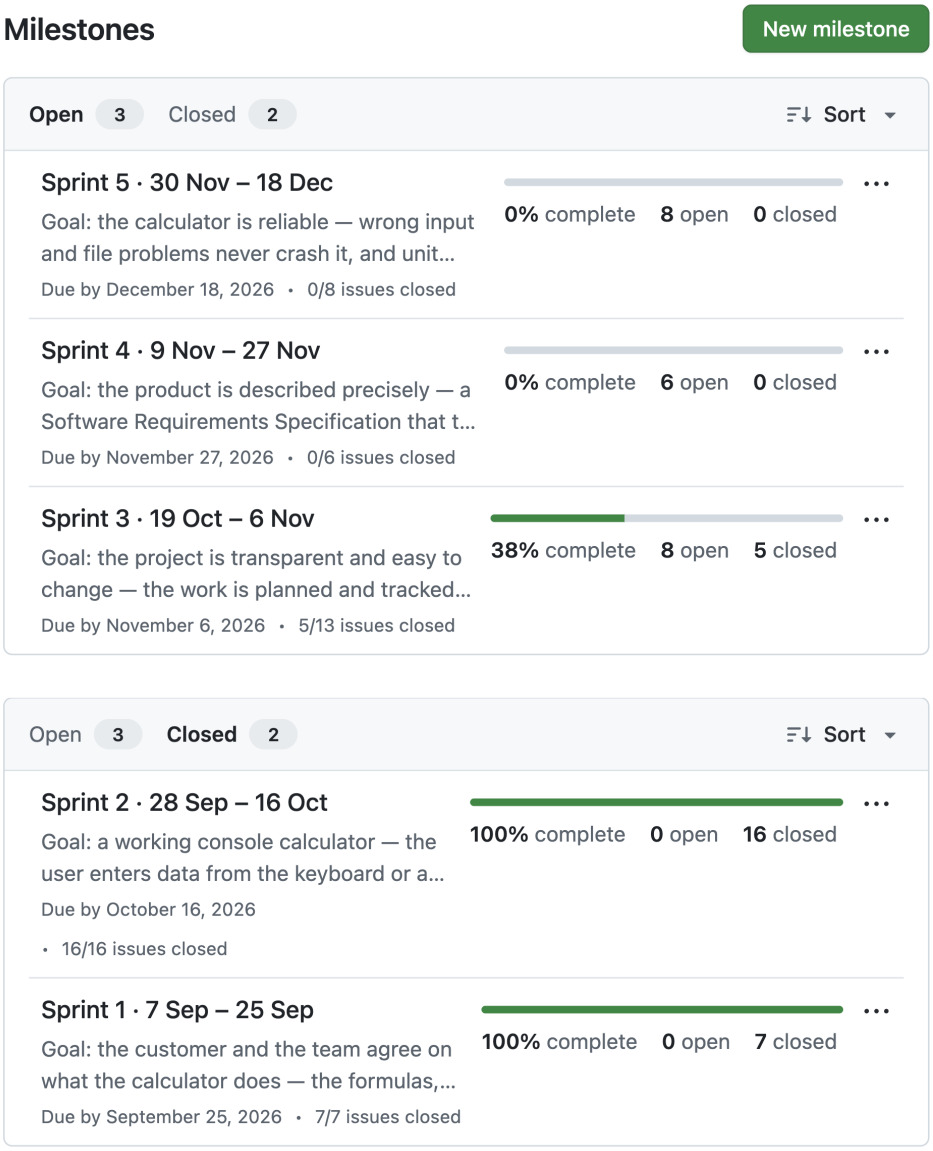



Figure 3.9 – Sprints of the project

An item is done when it meets the Definition of Done: its acceptance criteria are met, the code builds without warnings, the regression test passes, the change is merged into the main branch through a pull request and the documentation is updated. Sprints 1 and 2 are finished: 26 story points, all their stories and tasks are closed, so the milestones show 100 % (Figure 3.10).

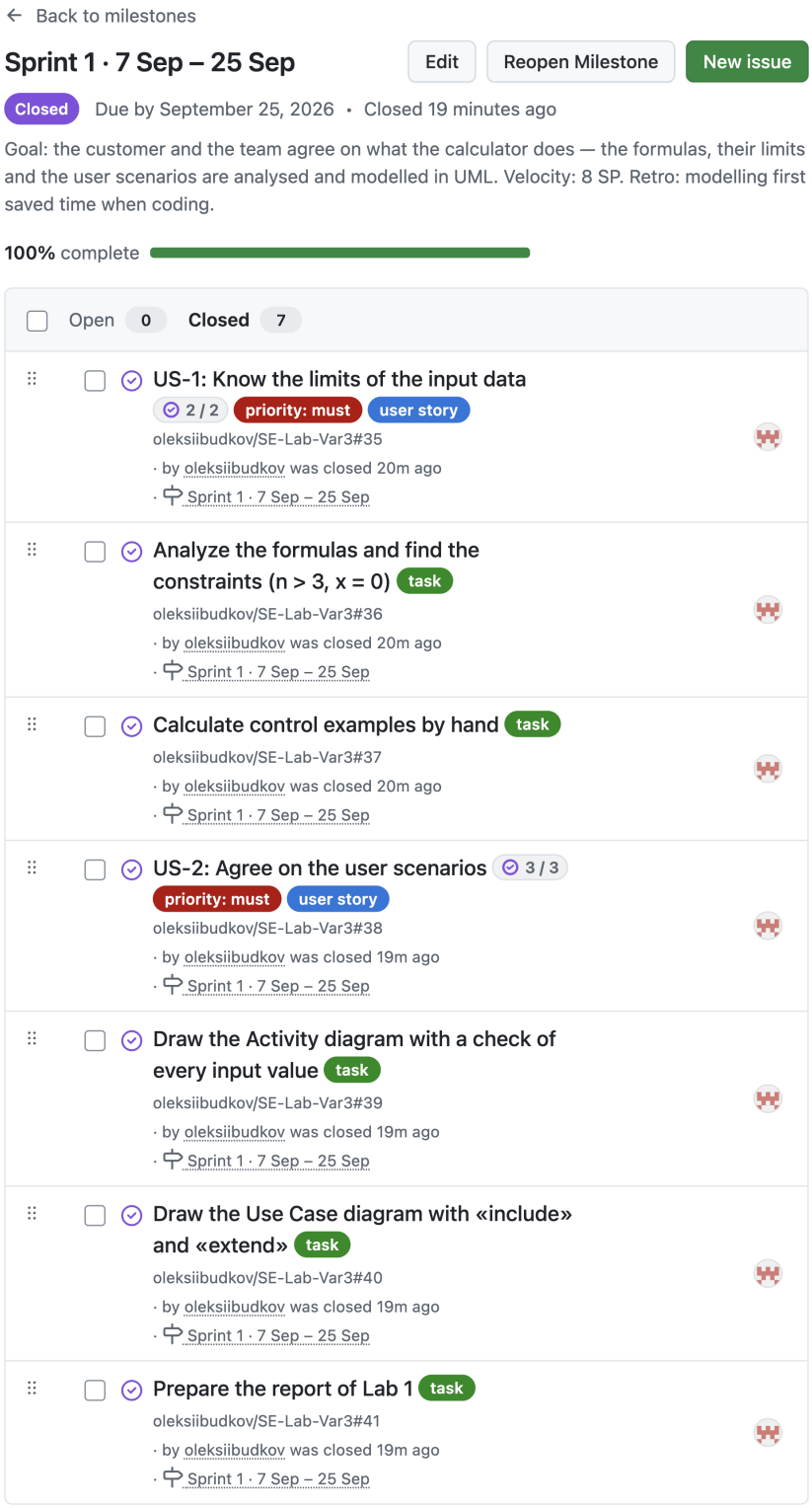



Figure 3.10 – Sprint 1 with its finished stories and tasks

The Scrum board is a GitHub Project linked to the repository. Its field Status has the columns Product Backlog, Sprint Backlog, In Progress and Done, and the fields Story Points and Priority hold the estimate. The view “Product Backlog” groups the stories by sprints and sums their story points (Figure 3.11); the view “Sprint 3 board” shows the current sprint as columns (Figure 3.12). The labels, sprints, stories, tasks and the board are created by the script tools/github_setup.sh with GitHub CLI, so the structure can be built again with one command.

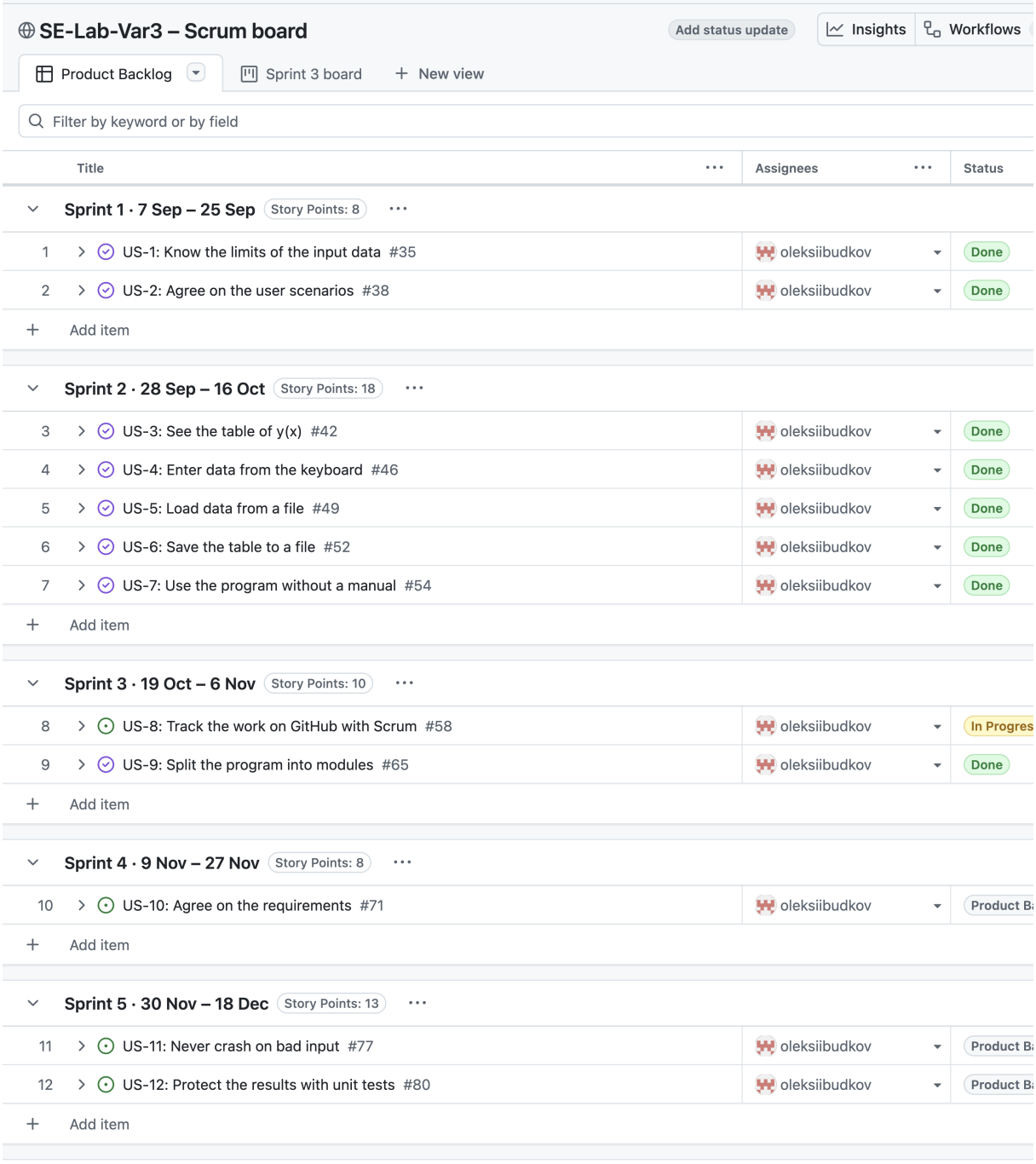



Figure 3.11 – Product Backlog view of the Scrum board

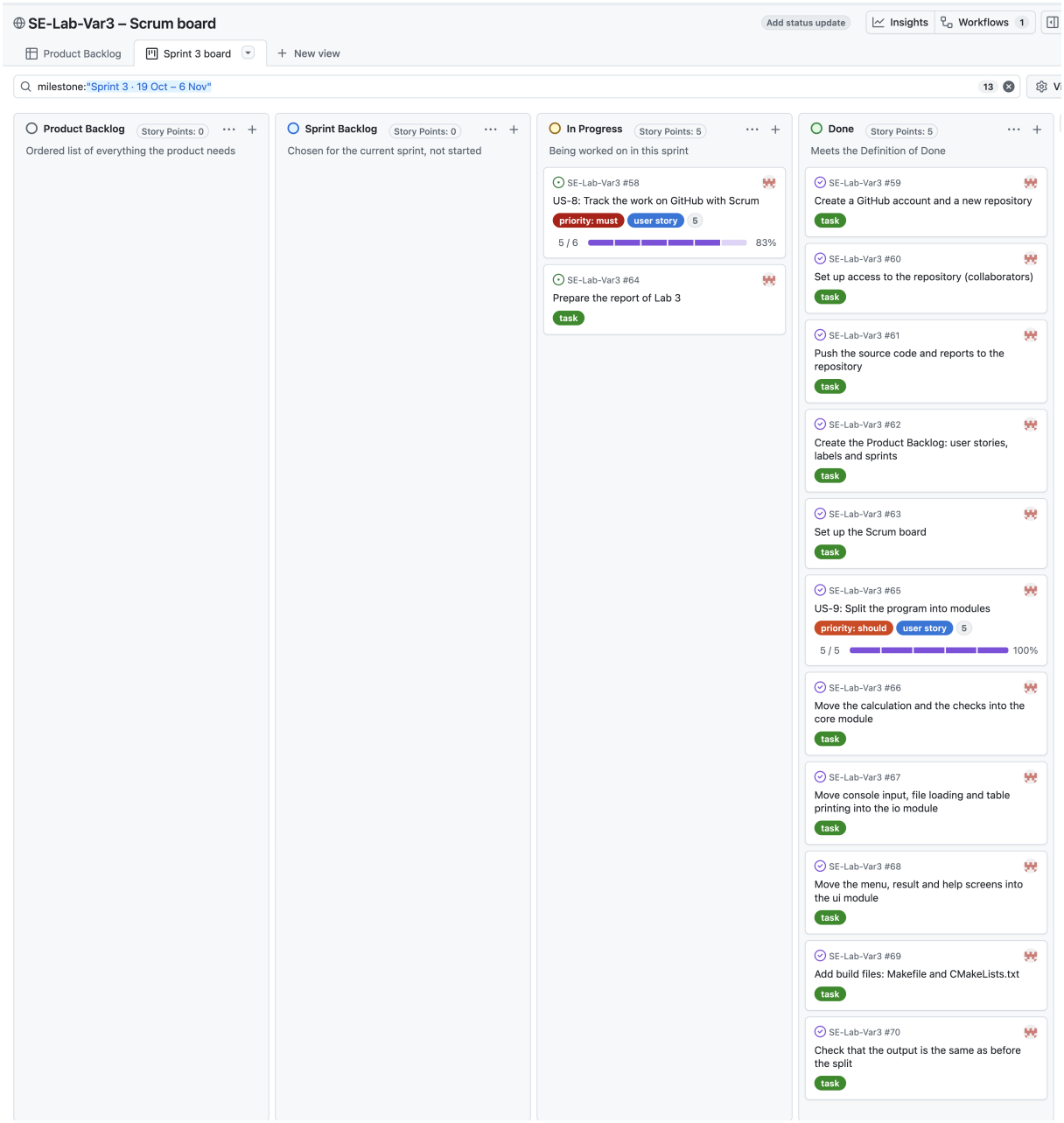



Figure 3.12 – Board of the current sprint

The stories and tasks of every sprint are shown in Tables 3.1–3.5, and the items that are not planned yet — in Table 3.6.

Table 3.1 – Sprint 1, 7.09–25.09.2026. Goal: the behaviour of the calculator is agreed and modelled (8 story points)

| User story | Task | Assignee | Status |
|---|---|---|---|
| US-1: Know the limits of the input data (#35)<br>3 SP, Must have | Analyze the formulas and find the constraints (n > 3, x = 0) (#36) | Budkov O. | Done |
| | Calculate control examples by hand (#37) | Budkov O. | Done |
| US-2: Agree on the user scenarios (#38)<br>5 SP, Must have | Draw the Activity diagram with a check of every input value (#39) | Budkov O. | Done |
| | Draw the Use Case diagram with «include» and «extend» (#40) | Budkov O. | Done |
| | Prepare the report of Lab 1 (#41) | Budkov O. | Done |

Table 3.2 – Sprint 2, 28.09–16.10.2026. Goal: a working console calculator (18 story points)

| User story | Task | Assignee | Status |
|---|---|---|---|
| US-3: See the table of y(x) (#42)<br>5 SP, Must have | Implement the calculation of y for both branches (#43) | Budkov O. | Done |
| | Print the result table with “not defined” and “too large” (#44) | Budkov O. | Done |
| | Compare the results with the control examples (#45) | Budkov O. | Done |
| US-4: Enter data from the keyboard (#46)<br>5 SP, Must have | Implement keyboard input with a separate check of every value (#47) | Budkov O. | Done |
| | Write clear error messages with examples (#48) | Budkov O. | Done |
| US-5: Load data from a file (#49)<br>3 SP, Should have | Implement loading of data from a file (#50) | Budkov O. | Done |
| | Keep the previous data when the file is wrong (#51) | Budkov O. | Done |
| US-6: Save the table to a file (#52)<br>2 SP, Should have | Implement saving of the table to results.txt (#53) | Budkov O. | Done |
| US-7: Use the program without a manual (#54)<br>3 SP, Should have | Add the main menu and the Help screen (#55) | Budkov O. | Done |
| | Create the wireframe and the diagram of screen transitions (#56) | Budkov O. | Done |
| | Prepare the report of Lab 2 (#57) | Budkov O. | Done |

Table 3.3 – Sprint 3, 19.10–6.11.2026. Goal: a transparent and modular project (10 story points)

| User story | Task | Assignee | Status |
|---|---|---|---|
| US-8: Track the work on GitHub with Scrum (#58)<br>5 SP, Must have | Create a GitHub account and a new repository (#59) | Budkov O. | Done |
| | Set up access to the repository (collaborators) (#60) | Budkov O. | Done |
| | Push the source code and reports to the repository (#61) | Budkov O. | Done |
| | Create the Product Backlog: user stories, labels and sprints (#62) | Budkov O. | Done |
| | Set up the Scrum board (#63) | Budkov O. | Done |
| | Prepare the report of Lab 3 (#64) | Budkov O. | Done |
| US-9: Split the program into modules (#65)<br>5 SP, Should have | Move the calculation and the checks into the core module (#66) | Budkov O. | Done |
| | Move console input, file loading and table printing into the io module (#67) | Budkov O. | Done |
| | Move the menu, result and help screens into the ui module (#68) | Budkov O. | Done |
| | Add build files: Makefile and CMakeLists.txt (#69) | Budkov O. | Done |
| | Check that the output is the same as before the split (#70) | Budkov O. | Done |

Table 3.4 – Sprint 4, 9.11–27.11.2026. Goal: the requirements are specified (8 story points)

| User story | Task | Assignee | Status |
|---|---|---|---|
| US-10: Agree on the requirements (#71)<br>8 SP, Must have | Choose a template for the SRS (#72) | Budkov O. | To do |
| | Write the purpose, definitions and system overview (#73) | Budkov O. | To do |
| | Write the overall description of the program (#74) | Budkov O. | To do |
| | Write the functional and non-functional requirements (#75) | Budkov O. | To do |
| | Prepare the report of Lab 4 (#76) | Budkov O. | To do |

Table 3.5 – Sprint 5, 30.11–18.12.2026. Goal: a reliable and tested calculator (13 story points)

| User story | Task | Assignee | Status |
|---|---|---|---|
| US-11: Never crash on bad input (#77)<br>5 SP, Must have | Add exception handling for wrong input and files (#78) | Budkov O. | To do |
| | Show one clear message for every type of error (#79) | Budkov O. | To do |
| US-12: Protect the results with unit tests (#80)<br>8 SP, Must have | Set up a unit test framework (#81) | Budkov O. | To do |
| | Write unit tests for the calculation of y (#82) | Budkov O. | To do |
| | Write unit tests for the input checks (#83) | Budkov O. | To do |
| | Prepare the report of Lab 5 (#84) | Budkov O. | To do |

Table 3.6 – Product Backlog items that are not planned for a sprint yet

| User story | Task | Assignee | Status |
|---|---|---|---|
| US-13: Choose the precision of the table (#85)<br>2 SP, Could have | not split into tasks yet | Budkov O. | To do |
| US-14: Export the table to CSV (#86)<br>3 SP, Could have | not split into tasks yet | Budkov O. | To do |

## Task 3.5 Working with the Git Repository

The source code and the reports were added to the repository from the command line in the folder of the project: git init, git add ., git commit -m "Add code and reports of Labs 1-3", git branch -M main, git remote add origin https://github.com/oleksiibudkov/SE-Lab-Var3.git and git push -u origin main (Figures 3.13–3.14). After that, every change is made in small steps: the changed files are staged with git add, a commit with a short message is created with git commit and pushed to github.com with git push (Figures 3.15–3.16). A separate branch is used for a separate piece of work: the branch lab3-planning was created with git switch -c and pushed with git push -u origin lab3-planning (Figure 3.17).

The user story US-9 “Split the program into modules” was made in the branch feature/modular-structure. The single file main.cpp was split into three layers: core — the data, the calculation of y and the input checks without any input or output; io — console input, loading from a file and printing the table; ui — the menu, result and help screens. main.cpp keeps only the loop of the main menu. The program is built with a Makefile or with CMake (Visual Studio), and a regression test (make test) checks that the output is the same as

before the split. Every commit names its task, for example “Move the screens into the ui module … Closes #68”, so GitHub closes the task when the branch is merged. The branch was merged into main through the pull request #87 (Figure 3.18); the history of the commits shows the merge (Figure 3.19).

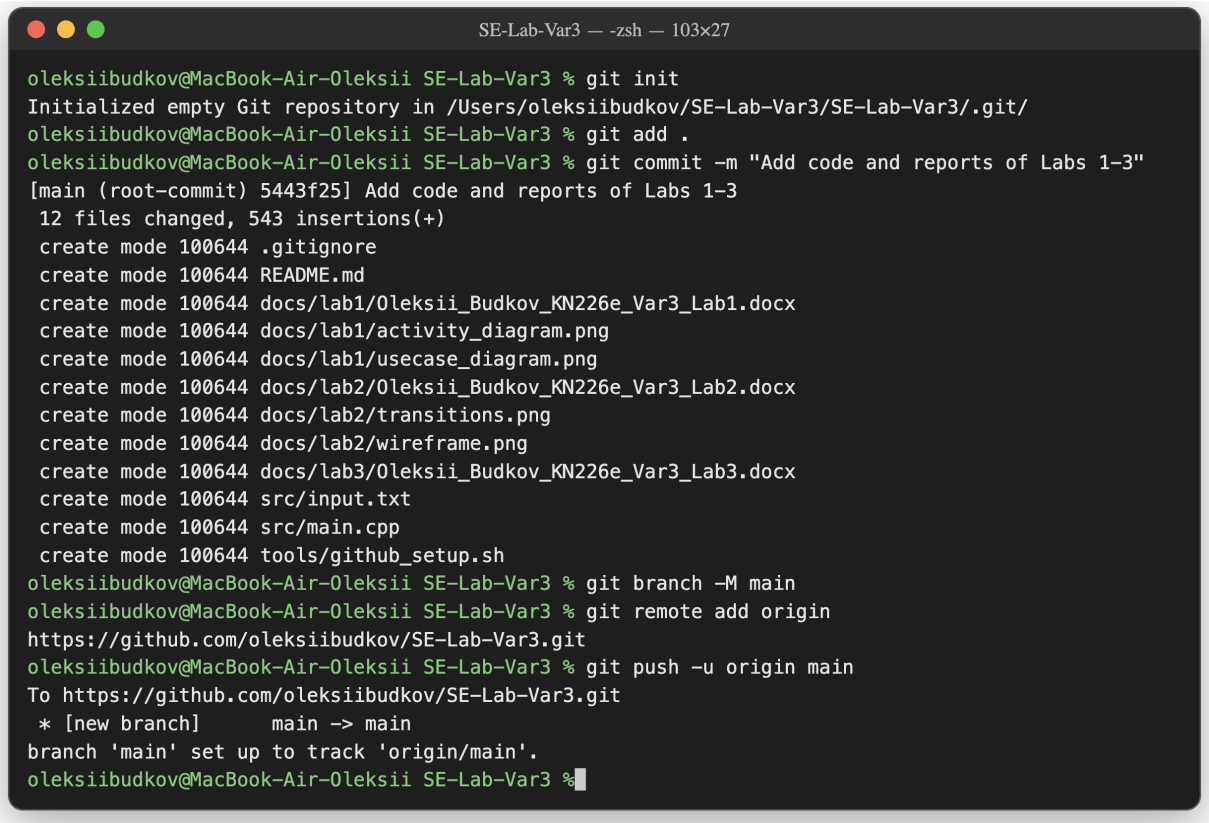



Figure 3.13 – Creating of a Git repository from the command line

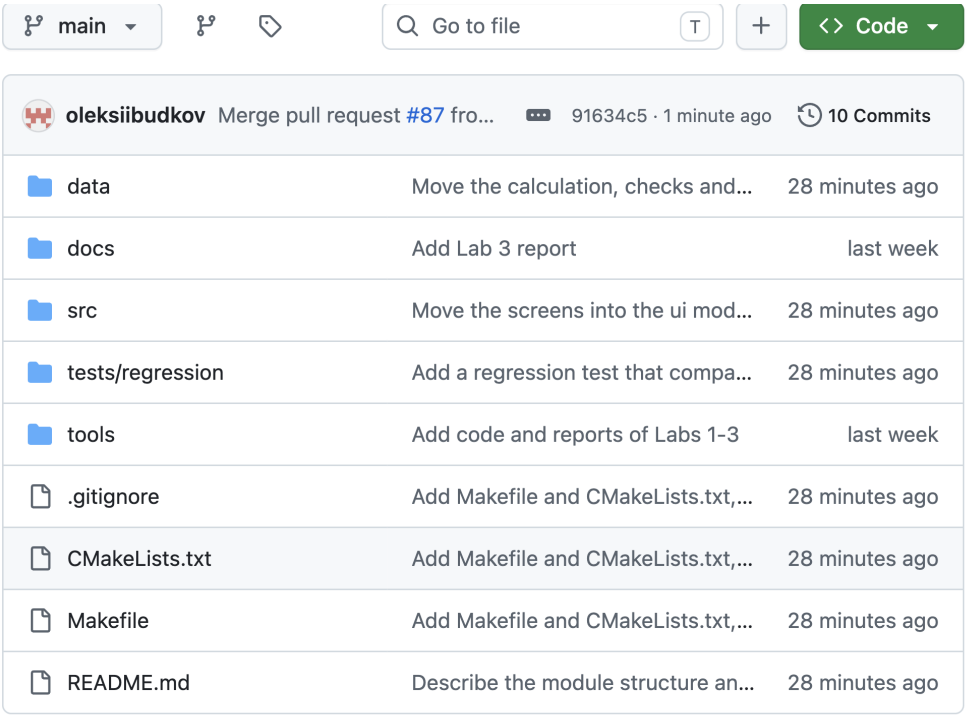



Figure 3.14 – Files of the project on github.com

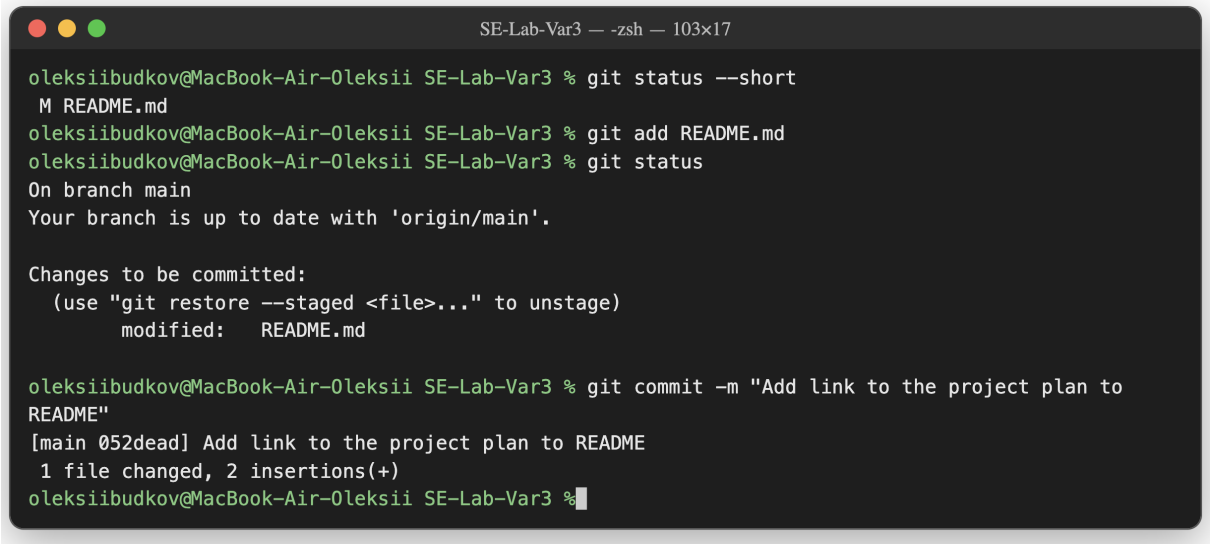



Figure 3.15 – Committing of the changed file

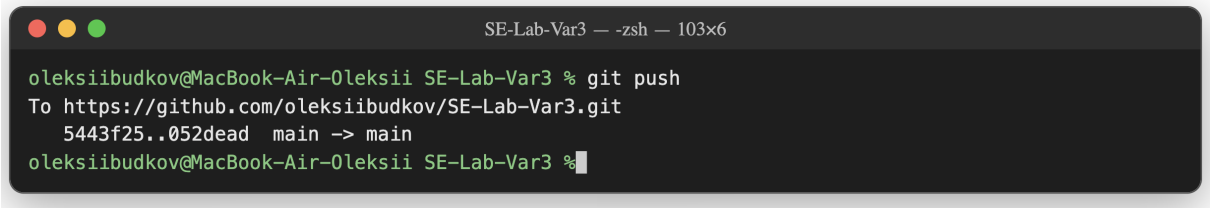



Figure 3.16 – Pushing of the commit

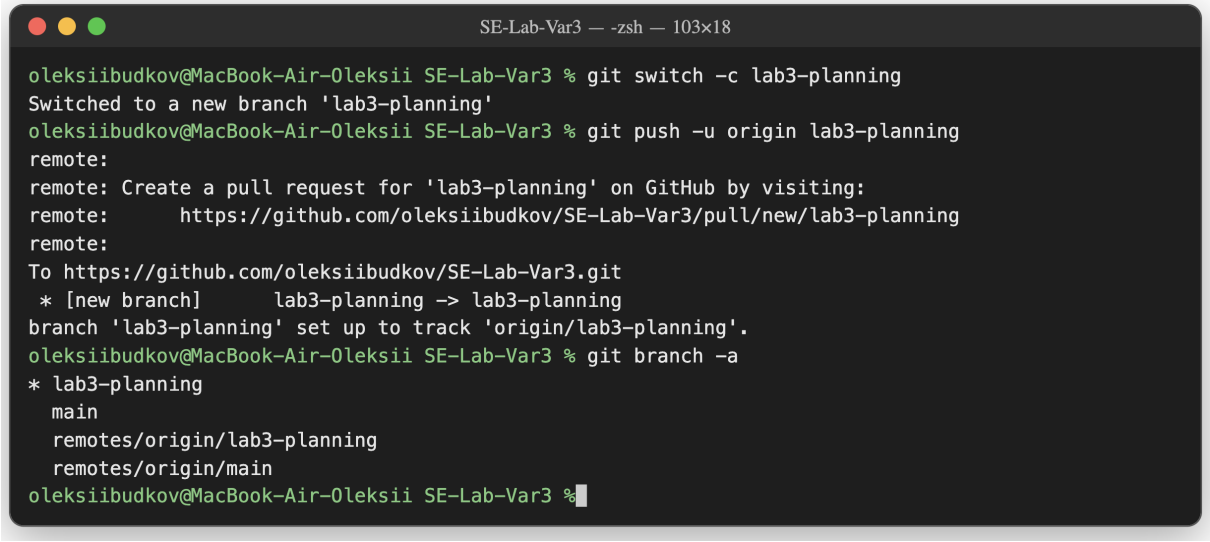



Figure 3.17 – Creating of a new branch

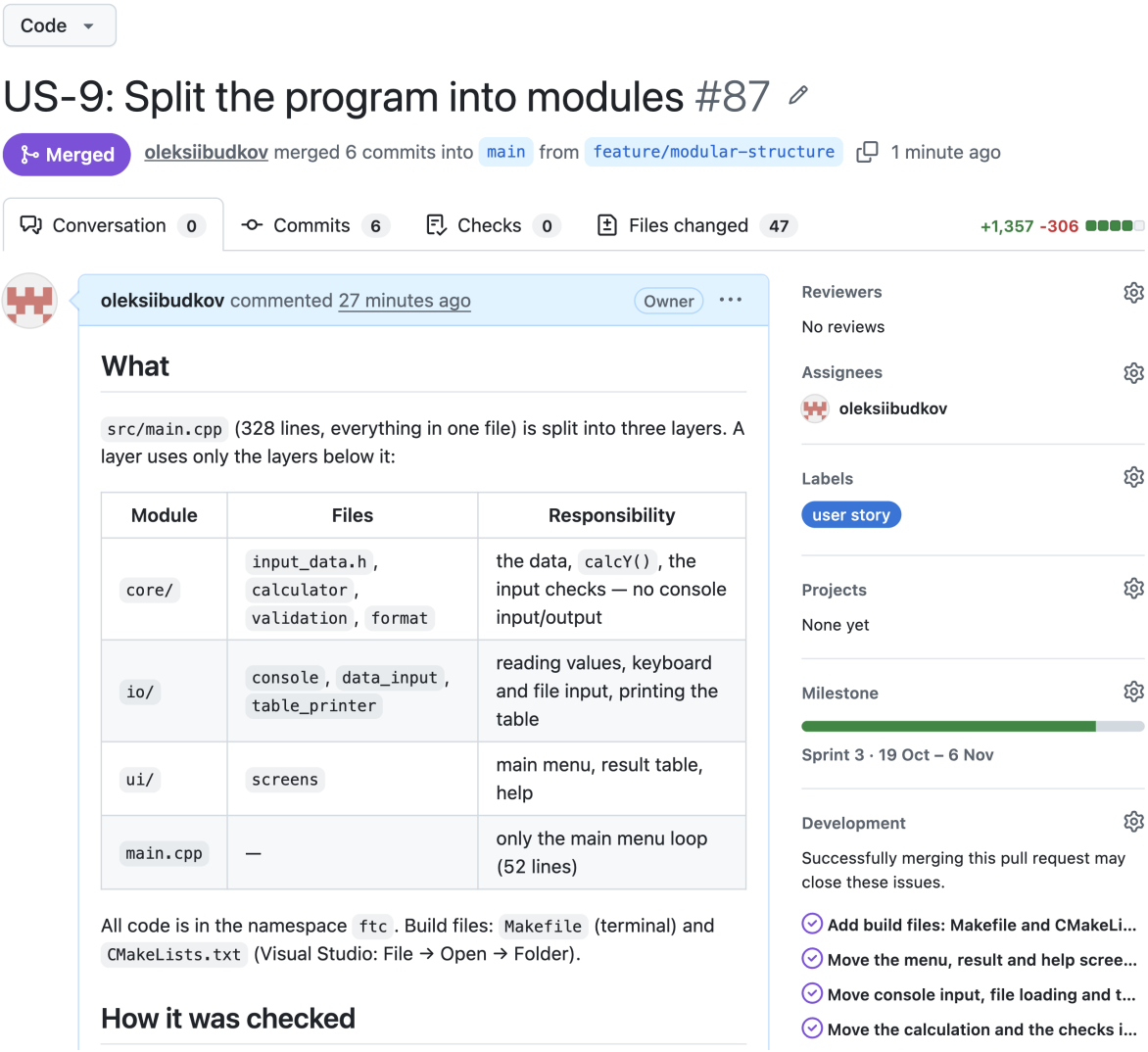



Figure 3.18 – Pull request of the user story US-9

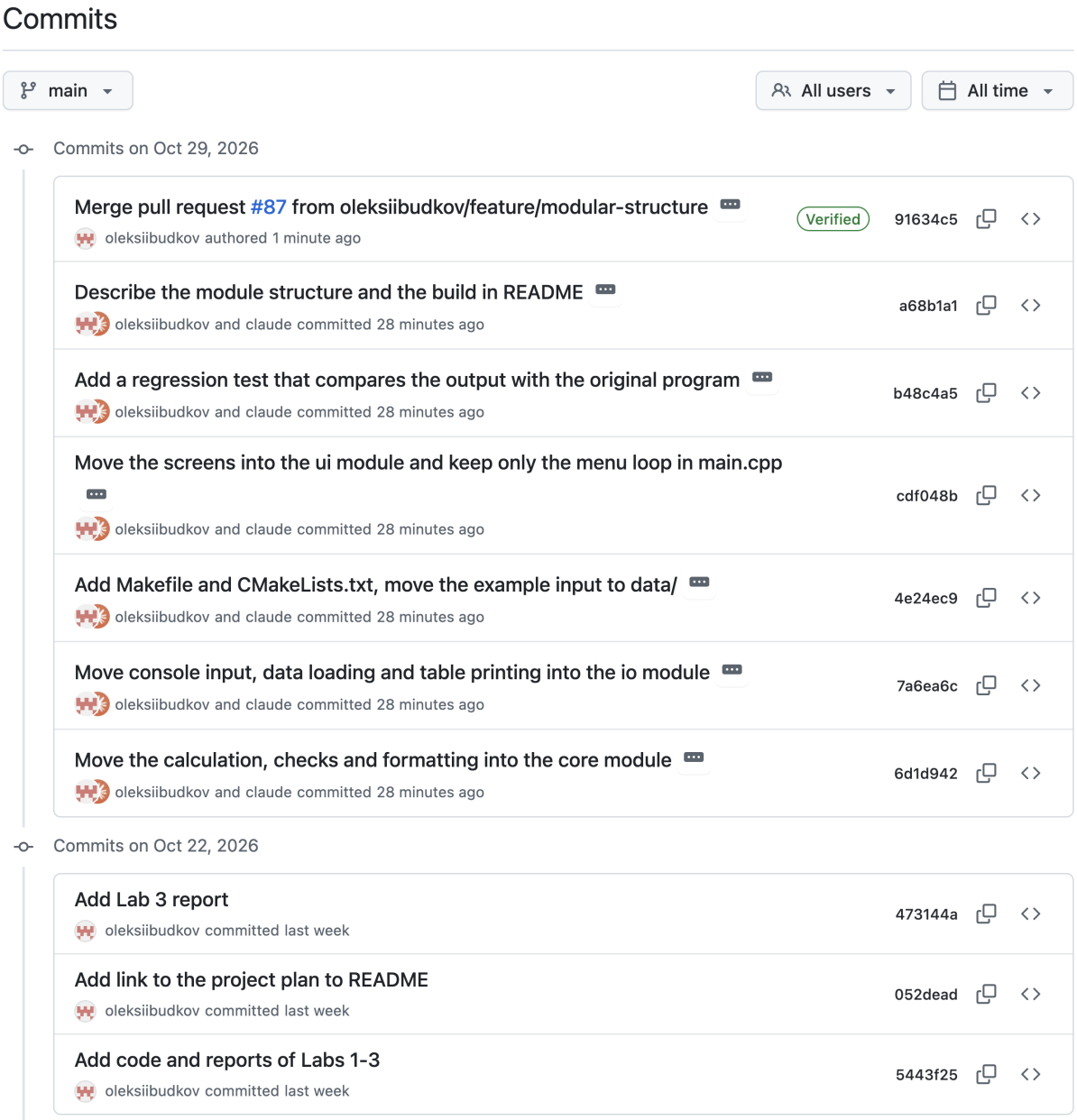



Figure 3.19 – History of commits

## Conclusions

During this laboratory work we got skills and knowledge about working with the project tracking system GitHub. We registered an account, created a repository for the project and learned how to add collaborators to it. We planned the work of all laboratory works with Scrum: a Product Backlog of user stories with acceptance criteria, story points and MoSCoW priorities, five three-week sprints with sprint goals, tasks as sub-issues of their stories and a Scrum board with the columns Product Backlog, Sprint Backlog, In Progress and Done. We stored the code and the reports in the git repository from the command line, split the program into modules in a feature branch and merged it into the main branch through a pull request that closed the user story and its tasks automatically.